My Project

Generated by Doxygen 1.9.8

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 4

Topic Documentation

4.1 XLIO Ultra API

High-performance zero-copy networking interface.

Modules

- Initialization and Cleanup

  Functions for initializing and cleaning up the XLIO Ultra API.
- Polling Groups

  Functions for managing polling groups and event handling.
- Socket Management

  Functions for creating and managing XLIO sockets.
- Transmit Operations

  High-performance data transmission functions.
- Receive Operations

  Zero-copy receive buffer management.
- Event Callbacks

  Callback functions for handling socket events.

4.1.1 Detailed Description

High-performance zero-copy networking interface.

The XLIO Ultra API is a performance-oriented, event-based networking interface designed for applications requiring maximum throughput and minimal latency. It provides zero-copy capabilities and efficient memory management for high-performance networking.

4.1.2 Key Features

- Native zero-copy TX and RX operations
- Immediate callback-based event notification without accumulation
- Optimized data path with fewer corner cases and non-performance-oriented features
  - No blocking mode
  - No partial writes
- Simplified application development:
  - Fewer error conditions and corner cases to handle
  - Flexible control of TX data aggregation
  - High-level TX operation completion
- Concurrency and scalability:
  - No global namespace - sockets grouped by polling groups
  - Clear concurrency boundaries and scalability model
  - Each group can be handled independently by different threads

4.1.3 Architecture Overview

The API is built around three main concepts:

1. **Polling Groups**: Event management and callback registration
2. **Sockets**: TCP socket abstraction with zero-copy capabilities
3. **Buffers**: Memory management for zero-copy operations

4.1.4 Typical Workflow

1. Initialize XLIO with xlio_init_ex()
2. Create polling group with xlio_poll_group_create()
3. Create socket with xlio_socket_create()
4. Configure socket (bind, connect, listen)
5. Poll for events with xlio_poll_group_poll()
6. Handle events via registered callbacks
7. Send/receive data using zero-copy operations
8. Clean up resources

4.1.5 Concurrency and Thread Safety

The XLIO Ultra API is designed for high-performance applications with specific concurrency patterns and thread safety requirements.

4.1.5.1 Thread Safety Model

- **The API is NOT thread-safe by default**
- Applications are responsible for proper serialization when accessing XLIO objects from multiple threads
- No internal locking is provided to maximize performance

4.1.5.2 Polling Group Concurrency

Polling groups are the primary mechanism for achieving concurrency:

- **Multiple polling groups can be polled concurrently** from different threads
- **Polling groups do not share resources** with each other
- **Sockets from different groups can be handled concurrently** without serialization

4.1.5.3 Serialization Requirements

- **Within a polling group**: All operations require serialization
  - Only one thread should call xlio_poll_group_poll() per group at a time
  - Socket operations within the same group must be serialized
  - Serialized polling group and socket calls can be executed by different threads
- **Across polling groups**: No serialization required
  - Different threads can operate on different groups simultaneously

4.1.5.4 Thread Safety Exceptions

Some operations have specific thread safety characteristics:

- **Initialization**: xlio_init_ex() and xlio_exit() are not thread-safe
- A group created with the flag XLIO_GROUP_FLAG_SAFE can execute a polling and socket TX operations concurrently

4.1.6 Current Limitations

- TCP sockets only (no UDP support)
- No crypto offload support
- No bonding support
- Only busy polling is supported
- fork() is supported only without created polling groups

4.1.7 Initialization and Cleanup

Functions for initializing and cleaning up the XLIO Ultra API.

Classes

- struct xlio_init_attr
  XLIO initialization attributes.

Functions

- int xlio_init_ex (const struct xlio_init_attr *attr)
  Initialize the XLIO Ultra API.
- int xlio_init (void)
  Initialize XLIO.
- int xlio_exit (void)
  Finalize XLIO.

4.1.7.1 Detailed Description

Functions for initializing and cleaning up the XLIO Ultra API.

4.1.7.2 Function Documentation

4.1.7.2.1 xlio_exit()

```
int xlio_exit (
            void  )
```

Finalize XLIO.

Finalizes and cleans XLIO resources.

Returns

0 on success, -1 on error (errno is set)

4.1.7.2.2 xlio_init()

```
int xlio_init (
            void  )
```

Initialize XLIO.

This function is similar to xlio_init_ex() but doesn't accept additional attributes.

Returns

0 on success, -1 on error (errno is set)

Error Codes:

- EINVAL: Invalid parameters
- ENOMEM: Insufficient memory
- ENODEV: No compatible network devices found
- EEXIST: XLIO is already initialized

See also

xlio_exit()

4.1.7.2.3 xlio_init_ex()

```
int xlio_init_ex (
            const struct xlio_init_attr * attr )
```

Initialize the XLIO Ultra API.

This function must be called before using any other XLIO Ultra API functions. It's a heavy operation that sets up the internal state, allocates resources, and configures the system for high-performance networking.

Note

This function is not thread-safe. However, subsequent serialized calls will exit successfully without performing any action.

Parameters

attr	Initialization attributes structure

Returns

0 on success, -1 on error (errno is set)

Error Codes:

- EINVAL: Invalid parameters
- ENOMEM: Insufficient memory
- ENODEV: No compatible network devices found

See also

xlio_exit()

xlio_init_attr

4.1.8 Polling Groups

Functions for managing polling groups and event handling.

Classes

- struct xlio_poll_group_attr

  Polling group attributes.

Macros

- #define XLIO_GROUP_FLAG_SAFE 0x1
- #define XLIO_GROUP_FLAG_DIRTY 0x2

Typedefs

- typedef uintptr_t xlio_poll_group_t

  Polling group handle.

Functions

- int xlio_poll_group_create (const struct xlio_poll_group_attr *attr, xlio_poll_group_t *group_out)

  Create a new polling group.
- int xlio_poll_group_destroy (xlio_poll_group_t group)

  Destroy a polling group.
- int xlio_poll_group_update (xlio_poll_group_t group, const struct xlio_poll_group_attr *attr)

  Update polling group attributes.
- void xlio_poll_group_poll (xlio_poll_group_t group)

  Poll for events on a polling group.

4.1.8.1 Detailed Description

Functions for managing polling groups and event handling.

Polling groups are the core event management mechanism in the XLIO Ultra API. They allow applications to register event callbacks and efficiently poll for network events across multiple sockets.

Polling group is a collection of sockets and resources required for their operation. Different polling groups can be used concurrently without serialization.

Polling groups provide loggical sockets organization for the following purposes:

- Achieving concurrency and scaling
- Implementing different RX / completion logic

Recommendations:

- Groups are expected to be long lived objects. Frequent creation/destruction has a penalty.
- Reduce the number of different network interfaces within a group to minimum. This will optimize the HW objects utilization. However, maintaining extra groups can have an overhead.

4.1.8.2 Macro Definition Documentation

4.1.8.2.1 XLIO_GROUP_FLAG_DIRTY

```
#define XLIO_GROUP_FLAG_DIRTY 0x2
```

Group will keep dirty sockets to be flushed with xlio_poll_group_flush().

4.1.8.2.2 XLIO_GROUP_FLAG_SAFE

```
#define XLIO_GROUP_FLAG_SAFE 0x1
```

Sockets and rings will be protected with locks regardless of XLIO configuration.

4.1.8.3 Typedef Documentation

4.1.8.3.1 xlio_poll_group_t

```
typedef uintptr_t xlio_poll_group_t
```

Polling group handle.

Opaque handle representing a polling group for event management.

4.1.8.4 Function Documentation

4.1.8.4.1 xlio_poll_group_create()

```
int xlio_poll_group_create (
            const struct xlio_poll_group_attr * attr,
            xlio_poll_group_t * group_out )
```

Create a new polling group.

Creates a new polling group with the specified attributes. Event callbacks are registered per group, allowing applications to implement different handling logic for different types of connections.

Parameters

attr	Polling group attributes
group_out	Pointer to store the created group handle

Returns

0 on success, -1 on error (errno is set)

Error Codes:

- EINVAL: Invalid parameters (group_out is NULL, attr is NULL, or socket_event_cb is NULL)
- ENOMEM: Insufficient memory

Note

socket_event_cb is mandatory.

See also

xlio_poll_group_destroy()

xlio_poll_group_attr

4.1.8.4.2 xlio_poll_group_destroy()

```
int xlio_poll_group_destroy (
            xlio_poll_group_t group )
```

Destroy a polling group.

Destroys the specified polling group and frees associated resources. All leftover sockets associated with this group are destroyed implicitly.

Parameters

group	The polling group to destroy

Returns

0 on success, -1 on error

4.1.8.4.3 xlio_poll_group_poll()

```
void xlio_poll_group_poll (
            xlio_poll_group_t group )
```

Poll for events on a polling group.

This is the main event processing function. It polls hardware for events, executes TCP timers, and invokes registered callbacks. Most network events are processed from the context of this call.

Parameters

group	The polling group to poll

Note

This function should be called regularly in the main event loop. It's non-blocking and will return immediately if no events are available.

4.1.8.4.4 xlio_poll_group_update()

```
int xlio_poll_group_update (
            xlio_poll_group_t group,
            const struct xlio_poll_group_attr * attr )
```

Update polling group attributes.

Updates the attributes of an existing polling group. This allows changing callback functions or flags without recreating the group.

Parameters

group	The polling group to update
attr	New attributes for the group

Returns

0 on success, -1 on error (errno is set)

Error Codes:

- EINVAL: Invalid parameters (attr is NULL or socket_event_cb is NULL)

4.1.9 Socket Management

Functions for creating and managing XLIO sockets.

Classes

- struct xlio_socket_attr

  Socket creation attributes.

Typedefs

- typedef uintptr_t xlio_socket_t

  Socket handle.

Functions

- int xlio_socket_create (const struct xlio_socket_attr *attr, xlio_socket_t *sock_out)

  Create a new XLIO socket.
- int xlio_socket_destroy (xlio_socket_t sock)

  Destroy an XLIO socket.
- int xlio_socket_update (xlio_socket_t sock, unsigned flags, uintptr_t userdata_sq)

  Update socket attributes.
- int xlio_socket_setsockopt (xlio_socket_t sock, int level, int optname, const void *optval, socklen_t optlen)

  Set socket options.
- int xlio_socket_getsockname (xlio_socket_t sock, struct sockaddr *addr, socklen_t *addrlen)

  Get socket name.
- int xlio_socket_getpeername (xlio_socket_t sock, struct sockaddr *addr, socklen_t *addrlen)

  Get peer name.
- int xlio_socket_bind (xlio_socket_t sock, const struct sockaddr *addr, socklen_t addrlen)

  Bind socket to address.
- int xlio_socket_connect (xlio_socket_t sock, const struct sockaddr *to, socklen_t tolen)

  Connect socket to remote address.
- int xlio_socket_listen (xlio_socket_t sock)

  Listen for incoming connections.
- struct ibv_pd * xlio_socket_get_pd (xlio_socket_t sock)

  Get InfiniBand protection domain.
- int xlio_socket_detach_group (xlio_socket_t sock)

  Detach socket from polling group.
- int xlio_socket_attach_group (xlio_socket_t sock, xlio_poll_group_t group)

  Attach socket to polling group.

4.1.9.1 Detailed Description

Functions for creating and managing XLIO sockets.

XLIO sockets are high-performance TCP socket abstractions that provide zero-copy capabilities. They are represented by opaque handles rather than file descriptors.

4.1.9.2 Typedef Documentation

4.1.9.2.1 xlio_socket_t

```
typedef uintptr_t xlio_socket_t
```

Socket handle.

Opaque handle representing an XLIO high-performance socket.

4.1.9.3 Function Documentation

4.1.9.3.1 xlio_socket_attach_group()

```
int xlio_socket_attach_group (
            xlio_socket_t sock,
            xlio_poll_group_t group )
```

Attach socket to polling group.

Attaches a previously detached socket to a polling group. The socket will begin generating events according to the group's configuration.

Parameters

sock	The socket to attach
group	The polling group to attach to

Returns

0 on success, -1 on error

Error Codes:

- EINVAL: Socket is already attached
- ENOMEM: No memory to complete the operation
- ENOTCONN: Failed to attach TX flow
- ECONNABORTED: Failed to attach RX flow

4.1.9.3.2 xlio_socket_bind()

```
int xlio_socket_bind (
            xlio_socket_t sock,
            const struct sockaddr * addr,
            socklen_t addrlen )
```

Bind socket to address.

Binds the socket to a local address, similar to bind().

Parameters

sock	The socket to bind
addr	The address to bind to
addrlen	Length of the address

Returns

0 on success, -1 on error (errno is set)

Error Codes:

- ENODEV: Trying to bind to a non-NVIDIA NIC
- Inherits bind(2) error codes

See also

bind(2)

4.1.9.3.3 xlio_socket_connect()

```
int xlio_socket_connect (
            xlio_socket_t sock,
            const struct sockaddr * to,
            socklen_t tolen )
```

Connect socket to remote address.

Initiates a connection to a remote address. The operation is non-blocking, and the connection status is reported via the socket event callback.

Parameters

sock	The socket to connect
to	The remote address to connect to
tolen	Length of the remote address

Returns

0 on success, -1 on error (errno is set)

Error Codes:

- EISCONN: The socket is already connected
- EALREADY: Previous connect attempt hasn't been completed yet
- ECONNABORTED: Previous connect attempt has failed
- ENODEV: Cannot establish connection with XLIO Ultra API or NVIDIA NIC

Note

This function returns immediately. Connection establishment is indicated by the XLIO_SOCKET_EVENT_↩ ESTABLISHED event. If a connection failure occurs an XLIO_SOCKET_EVENT_ERROR event will be delivered.

See also

connect(2)

4.1.9.3.4 xlio_socket_create()

```
int xlio_socket_create (
            const struct xlio_socket_attr * attr,
            xlio_socket_t * sock_out )
```

Create a new XLIO socket.

Creates a new XLIO socket with the specified attributes. The socket is automatically associated with the specified polling group and configured for high-performance operation.

Parameters

attr	Socket attributes
sock_out	Pointer to store the created socket handle

Returns

0 on success, -1 on error (errno is set)

Error Codes:

- EINVAL: Invalid parameters (sock_out is NULL, attr is NULL, group is invalid, or domain is not AF_↩ INET/AF_INET6)
- ENOMEM: Insufficient memory
- EMFILE: Too many open files

See also

xlio_socket_destroy()

xlio_socket_attr

4.1.9.3.5 xlio_socket_destroy()

```
int xlio_socket_destroy (
            xlio_socket_t sock )
```

Destroy an XLIO socket.

Initiates the socket closing procedure. The process may be asynchronous, and socket events may continue to arrive until the XLIO_SOCKET_EVENT_TERMINATED event is received.

Parameters

sock	The socket to destroy

Returns

0 on success, -1 on error (errno is set)

Error Codes:

- EINVAL: Invalid socket handle

Note

Zero-copy completion events may still arrive after calling this function until the TERMINATED event is received.

4.1.9.3.6 xlio_socket_detach_group()

```
int xlio_socket_detach_group (
            xlio_socket_t sock )
```

Detach socket from polling group.

Removes the socket from its current polling group. The socket becomes inactive and will not generate events until attached to another group.

Parameters

sock	The socket to detach

Returns

0 on success, -1 on error

Error Codes:

- EINVAL: Socket is not connected or already detached
- ENOTSUP: Not supported with listen sockets

Note

During the 2-step socket migration (detach -> attach), there is a time window during which RX packets are dropped until the socket is completely attached to the new group. Applications should minimize this window to avoid packet loss and TCP retransmissions.

4.1.9.3.7 xlio_socket_get_pd()

```
struct ibv_pd * xlio_socket_get_pd (
            xlio_socket_t sock )
```

Get InfiniBand protection domain.

Returns the InfiniBand protection domain associated with the socket. This can be used for registering memory regions for zero-copy operations.

Parameters

sock	The socket to query

Returns

Pointer to ibv_pd structure, or NULL on error

Note

Socket must be connected or in progress of connecting.

4.1.9.3.8 xlio_socket_getpeername()

```
int xlio_socket_getpeername (
            xlio_socket_t sock,
            struct sockaddr * addr,
            socklen_t * addrlen )
```

Get peer name.

Retrieves the remote address of the socket, similar to getpeername().

Parameters

sock	The socket to query
addr	Buffer to store the address
addrlen	Pointer to the address length

Returns

0 on success, -1 on error (errno is set)

See also

getpeername(2)

4.1.9.3.9 xlio_socket_getsockname()

```
int xlio_socket_getsockname (
            xlio_socket_t sock,
            struct sockaddr * addr,
            socklen_t * addrlen )
```

Get socket name.

Retrieves the local address of the socket, similar to getsockname().

Parameters

sock	The socket to query
addr	Buffer to store the address
addrlen	Pointer to the address length

Returns

0 on success, -1 on error (errno is set)

See also

getsockname(2)

4.1.9.3.10 xlio_socket_listen()

```
int xlio_socket_listen (
            xlio_socket_t sock )
```

Listen for incoming connections.

Configures the socket to listen for incoming connections. Requires that the polling group has a socket_accept_cb callback registered.

Parameters

sock	The socket to configure for listening

Returns

0 on success, -1 on error (errno is set)

Error Codes:

- ENOTCONN: No accept callback registered in the polling group
- EINVAL: Socket is already connected
- EADDRINUSE: Another socket is already listening on the same port
- ENODEV: Trying to listen on a non-NVIDIA NIC or an internal error preventing the socket from being offloaded

Note

The socket must be bound before calling this function.

See also

listen(2)

4.1.9.3.11 xlio_socket_setsockopt()

```
int xlio_socket_setsockopt (
            xlio_socket_t sock,
            int level,
            int optname,
            const void * optval,
            socklen_t optlen )
```

Set socket options.

Sets socket options, similar to the standard setsockopt() function. Supports standard socket options as well as XLIO-specific options.

Parameters

sock	The socket to configure
level	The protocol level (SOL_SOCKET, IPPROTO_TCP, etc.)
optname	The option name
optval	Pointer to the option value
optlen	Length of the option value

Returns

0 on success, -1 on error (errno is set)

See also

setsockopt(2)

4.1.9.3.12 xlio_socket_update()

```
int xlio_socket_update (
            xlio_socket_t sock,
```

```
            unsigned flags,
            uintptr_t userdata_sq )
```

Update socket attributes.

Updates the flags and user data associated with a socket. This allows changing socket behavior and context without recreating the socket.

Parameters

sock	The socket to update
flags	New flags for the socket
userdata_sq	New user data for the socket

Returns

0 on success, -1 on error

4.1.10 Transmit Operations

High-performance data transmission functions.

Classes

- struct xlio_socket_send_attr

  Send operation attributes.

Macros

- #define XLIO_SOCKET_SEND_FLAG_FLUSH 0x1
- #define XLIO_SOCKET_SEND_FLAG_INLINE 0x2

Functions

- int xlio_socket_send (xlio_socket_t sock, const void ∗data, size_t len, const struct xlio_socket_send_attr ∗attr)

  Send data on a socket.
- int xlio_socket_sendv (xlio_socket_t sock, const struct iovec ∗iov, unsigned iovcnt, const struct xlio_socket_send_attr ∗attr)

  Send vectored data on a socket.
- void xlio_poll_group_flush (xlio_poll_group_t group)

  Flush all dirty sockets in a polling group.
- void xlio_socket_flush (xlio_socket_t sock)

  Flush pending data on a socket.

4.1.10.1 Detailed Description

High-performance data transmission functions.

The XLIO Ultra API provides efficient transmission capabilities with zero-copy support and flexible batching options.

4.1.10.2 TX Flow Properties

- Non-blocking operation
- No partial write support - accepts all data unless memory allocation fails
- Zero-copy completion callbacks for memory management
- Inline send operations support for small data
- Data aggregation with explicit flush control

4.1.10.3 TX Flow Limitations

- Currently, data can be pushed to wire in the RX flow regardless of the flush logic
- Avoid using xlio_socket_flush() for a XLIO_GROUP_FLAG_DIRTY group
- For a XLIO_GROUP_FLAG_DIRTY group, usage of XLIO_SOCKET_SEND_FLAG_FLUSH is limited, it's better to avoid using them both simultaneously.

4.1.10.4 Macro Definition Documentation

4.1.10.4.1 XLIO_SOCKET_SEND_FLAG_FLUSH

```
#define XLIO_SOCKET_SEND_FLAG_FLUSH 0x1
```

Flush socket after queueing the data.

4.1.10.4.2 XLIO_SOCKET_SEND_FLAG_INLINE

```
#define XLIO_SOCKET_SEND_FLAG_INLINE 0x2
```

Copy user data to the internal buffers instead of taking ownership.

4.1.10.5 Function Documentation

4.1.10.5.1 xlio_poll_group_flush()

```
void xlio_poll_group_flush (
            xlio_poll_group_t group )
```

Flush all dirty sockets in a polling group.

For polling groups created with XLIO_GROUP_FLAG_DIRTY, this function flushes all sockets that have pending data to send. This provides batch flushing capabilities for improved performance.

Parameters

group	The polling group to flush

Note

This function should only be used with groups that have the XLIO_GROUP_FLAG_DIRTY flag set.

4.1.10.5.2 xlio_socket_flush()

```
void xlio_socket_flush (
            xlio_socket_t sock )
```

Flush pending data on a socket.

Forces transmission of any data queued on the socket. XLIO aggregates data by default for efficiency and user logic simplification.

This function doesn't guarantee immediate transmission, because TCP algorithms and congestion/flow control may affect transmission.

Parameters

sock	The socket to flush

Note

Avoid using this function with sockets in XLIO_GROUP_FLAG_DIRTY groups. Use xlio_poll_group_flush() instead for better performance for such groups.

4.1.10.5.3 xlio_socket_send()

```
int xlio_socket_send (
            xlio_socket_t sock,
            const void * data,
            size_t len,
            const struct xlio_socket_send_attr * attr )
```

Send data on a socket.

Sends data on the specified socket using zero-copy by default. The operation is non-blocking and accepts all data unless memory allocation fails.

Parameters

sock	The socket to send data on
data	Pointer to the data to send
len	Length of the data
attr	Send attributes controlling the operation

Returns

0 on success, -1 on error (errno is set)

Error Codes:

- ENOMEM: Insufficient memory (recoverable by retrying later)
- Other errors are generally not recoverable

Note

For zero-copy operation, the memory must be registered with the InfiniBand protection domain obtained from xlio_socket_get_pd().

See also

xlio_socket_send_attr

4.1.10.5.4 xlio_socket_sendv()

```
int xlio_socket_sendv (
            xlio_socket_t sock,
            const struct iovec * iov,
            unsigned iovcnt,
            const struct xlio_socket_send_attr * attr )
```

Send vectored data on a socket.

Sends data from multiple buffers (scatter-gather) on the specified socket.

Parameters

sock	The socket to send data on
iov	Array of iovec structures describing the data buffers
iovcnt	Number of iovec structures
attr	Send attributes controlling the operation

Returns

0 on success, -1 on error (errno is set)

See also

xlio_socket_send()

4.1.11 Receive Operations

Zero-copy receive buffer management.

Classes

- struct xlio_buf

  Buffer descriptor.

Functions

- void xlio_socket_buf_free (xlio_socket_t sock, struct xlio_buf *buf)

  Free a receive buffer (socket-specific)
- void xlio_poll_group_buf_free (xlio_poll_group_t group, struct xlio_buf *buf)

  Free a receive buffer (group-specific)

4.1.11.1 Detailed Description

Zero-copy receive buffer management.

The XLIO Ultra API provides zero-copy receive capabilities through a buffer management system. Received data is delivered via callbacks with buffer descriptors that must be returned to the system.

xlio_buf structure contains an uninitialized userdata field which can be used by the application to store any data during its ownership on the buffer. For example, the field can be used to organize a list without a container allocation, or to add a reference counter to the buffer.

4.1.11.2 Data Alignment Considerations

XLIO Ultra API does not guarantee alignment for zero-copy RX data. The data alignment depends on the underlying network headers and packet structure.

4.1.11.3 Function Documentation

4.1.11.3.1 xlio_poll_group_buf_free()

```
void xlio_poll_group_buf_free (
            xlio_poll_group_t group,
            struct xlio_buf * buf )
```

Free a receive buffer (group-specific)

Returns a receive buffer to the system for reuse. This function allows to return a buffer outside of the original socket lifecycle.

Parameters

group	The polling group
buf	The buffer descriptor to free

Note

The buffer must not be accessed after calling this function.

4.1.11.3.2 xlio_socket_buf_free()

```
void xlio_socket_buf_free (
```

```
            xlio_socket_t sock,
            struct xlio_buf * buf )
```

Free a receive buffer (socket-specific)

Returns a receive buffer to the system for reuse. This function should be called for every buffer received via the RX callback.

Parameters

sock	The socket that received the buffer
buf	The buffer descriptor to free

Note

The buffer must not be accessed after calling this function.

4.1.12 Event Callbacks

Callback functions for handling socket events.

Typedefs

- typedef void(∗ xlio_memory_cb_t) (void ∗addr, size_t len, size_t hugepage_size)

  Memory allocation callback function.
- typedef void(∗ xlio_socket_event_cb_t) (xlio_socket_t sock, uintptr_t userdata_sq, int event, int value)

  Socket event callback function.
- typedef void(∗ xlio_socket_comp_cb_t) (xlio_socket_t sock, uintptr_t userdata_sq, uintptr_t userdata_op)

  Zero-copy completion callback function.
- typedef void(∗ xlio_socket_rx_cb_t) (xlio_socket_t sock, uintptr_t userdata_sq, void ∗data, size_t len, struct xlio_buf ∗buf)

  Receive data callback function.
- typedef void(∗ xlio_socket_accept_cb_t) (xlio_socket_t sock, xlio_socket_t parent, uintptr_t parent_↩ userdata_sq)

  Accept callback function.

Enumerations

- enum { XLIO_SOCKET_EVENT_ESTABLISHED = 1 , XLIO_SOCKET_EVENT_TERMINATED , XLIO_SOCKET_EVENT_CLOSED , XLIO_SOCKET_EVENT_ERROR }

  Socket events.

4.1.12.1 Detailed Description

Callback functions for handling socket events.

The XLIO Ultra API uses callbacks to notify applications of various events including connection state changes, data arrival, and completion of zero-copy operations.

Most of the callbacks are expected from the xlio_poll_group_poll() context.

4.1.12.2 Typedef Documentation

4.1.12.2.1 xlio_memory_cb_t

```
typedef void(* xlio_memory_cb_t) (void *addr, size_t len, size_t hugepage_size)
```

Memory allocation callback function.

This callback is invoked when XLIO allocates memory regions that can be used for RX buffers. Applications can use this information for memory management or preparation.

Parameters

addr	Base address of the allocated memory
len	Size of the allocated memory
hugepage_size	Page size if hugepages are used, 0 for regular pages

Note

If hugepage_size is non-zero, both addr and len are aligned to the page size boundary. For external allocators, hugepage_size is always reported as zero.

See also

xlio_init_attr

4.1.12.2.2 xlio_socket_accept_cb_t

```
typedef void(* xlio_socket_accept_cb_t) (xlio_socket_t sock, xlio_socket_t parent, uintptr_t
parent_userdata_sq)
```

Accept callback function.

This callback is invoked when a new connection is accepted on a listening socket. The new socket is automatically created and associated with the same polling group.

Parameters

sock	The newly accepted socket
parent	The listening socket that accepted the connection
parent_userdata_sq	User data from the parent socket

Note

The new socket inherits the polling group from the parent but may need additional configuration (e.g., userdata_sq update).

See also

xlio_socket_update()

xlio_poll_group_attr

4.1.12.2.3 xlio_socket_comp_cb_t

```
typedef void(* xlio_socket_comp_cb_t) (xlio_socket_t sock, uintptr_t userdata_sq, uintptr_↩
t userdata_op)
```

Zero-copy completion callback function.

This callback is invoked when a zero-copy send operation completes, allowing the application to reclaim or reuse the transmitted buffers.

Parameters

sock	The socket that completed the operation
userdata_sq	User data associated with the socket
userdata_op	User data associated with the specific operation

Calling Contexts:

- xlio_poll_group_poll() (most common)
- xlio_socket_send() (if data is immediately flushed)
- xlio_socket_flush() / xlio_poll_group_flush()

Note

Send operations are allowed in this callback unless the socket is being destroyed.

See also

xlio_socket_send_attr

xlio_poll_group_attr

4.1.12.2.4 xlio_socket_event_cb_t

```
typedef void(* xlio_socket_event_cb_t) (xlio_socket_t sock, uintptr_t userdata_sq, int event,
int value)
```

Socket event callback function.

This callback is invoked when socket state changes occur, such as connection establishment, errors, or termination.

Parameters

sock	The socket generating the event
userdata_sq	User data associated with the socket
event	The event type (XLIO_SOCKET_EVENT_*)
value	Event-specific value (error code for ERROR events, 0 otherwise)

Event Types:

- XLIO_SOCKET_EVENT_ESTABLISHED: TCP connection established
- XLIO_SOCKET_EVENT_TERMINATED: Socket terminated, no further events
- XLIO_SOCKET_EVENT_CLOSED: Passive close by remote peer
- XLIO_SOCKET_EVENT_ERROR: Error occurred, see value for error code

Error Codes (for ERROR events):

- ECONNABORTED: Connection aborted by local side
- ECONNRESET: Connection reset by remote side
- ECONNREFUSED: Connection refused during handshake
- ETIMEDOUT: Connection timed out

Note

Send operations are allowed only from the ESTABLISHED event context.

See also

xlio_poll_group_attr

4.1.12.2.5 xlio_socket_rx_cb_t

```
typedef void(* xlio_socket_rx_cb_t) (xlio_socket_t sock, uintptr_t userdata_sq, void *data,
size_t len, struct xlio_buf *buf)
```

Receive data callback function.

This callback is invoked when TCP payload arrives on a socket. Each call provides a single contiguous buffer containing received data.

Parameters

sock	The socket that received the data
userdata_sq	User data associated with the socket
data	Pointer to the received data
len	Length of the received data
buf	Buffer descriptor that must be returned via xlio_*_buf_free()

Note

The data pointer is valid only until the buffer is freed. The buffer's userdata field can be used during user ownership.

See also

xlio_socket_buf_free()

xlio_poll_group_buf_free()

xlio_poll_group_attr

4.1.12.3 Enumeration Type Documentation

4.1.12.3.1 anonymous enum

```
anonymous enum
```

Socket events.

Enumerator

XLIO_SOCKET_EVENT_ESTABLISHED	TCP connection established.
XLIO_SOCKET_EVENT_TERMINATED	Socket terminated and no further events are possible.
XLIO_SOCKET_EVENT_CLOSED	Passive close.
XLIO_SOCKET_EVENT_ERROR	An error occurred, see the error code value.

Chapter 5

Class Documentation

5.1 xlio_buf Struct Reference

Buffer descriptor.

`#include <ultrapai.h>`

Public Attributes

- uint64_t **userdata**

5.1.1 Detailed Description

Buffer descriptor.

Opaque structure representing a receive buffer in zero-copy RX operations. Buffers are provided via RX callbacks and must be returned to XLIO.

Buffer Lifecycle:

1. Buffer provided to application via xlio_socket_rx_cb_t
2. Application processes data and optionally uses userdata field
3. Application returns buffer via xlio_socket_buf_free() or xlio_poll_group_buf_free()

User Data Field:

- Available for application use during buffer ownership
- Can be used for reference counting, linking, or other purposes
- Not initialized by XLIO

Structure Members:

- uint64_t userdata: User data field available during buffer ownership

The documentation for this struct was generated from the following file:

- ultrapai.h

5.2 xlio_init_attr Struct Reference

XLIO initialization attributes.

`#include <ultrapai.h>`

Public Attributes

- unsigned **flags**
- xlio_memory_cb_t **memory_cb**
- void ∗(∗ **memory_alloc**)(size_t)
- void(∗ **memory_free**)(void ∗)

5.2.1 Detailed Description

XLIO initialization attributes.

Structure containing parameters for XLIO initialization with xlio_init_ex().

Memory Management:

- memory_cb: Called when XLIO allocates memory regions for RX buffers
- memory_alloc/memory_free: Optional external allocator functions

External Allocator Notes:

- When external allocator is provided, XLIO uses it instead of internal allocation
- Current implementation allocates a single memory block during xlio_init_ex()
- For external allocators, hugepage_size in memory_cb is always reported as zero

Structure Members:

- unsigned flags: Initialization flags (reserved for future use)
- xlio_memory_cb_t memory_cb: Memory allocation notification callback
- void ∗(∗memory_alloc)(size_t): Optional external memory allocator function
- void (∗memory_free)(void ∗): Optional external memory deallocator function

The documentation for this struct was generated from the following file:

- ultrapai.h

5.3 xlio_poll_group_attr Struct Reference

Polling group attributes.

`#include <ultrapai.h>`

Public Attributes

- unsigned **flags**
- xlio_socket_event_cb_t **socket_event_cb**
- xlio_socket_comp_cb_t **socket_comp_cb**
- xlio_socket_rx_cb_t **socket_rx_cb**
- xlio_socket_accept_cb_t **socket_accept_cb**

5.3.1 Detailed Description

Polling group attributes.

Structure containing configuration for a polling group creation and updates. Event callbacks are registered per group, allowing different handling logic for different types of connections.

Required Callbacks:

- socket_event_cb: Must be provided (handles connection state changes)

Optional Callbacks:

- socket_comp_cb: Zero-copy completion notifications
- socket_rx_cb: Receive data notifications
- socket_accept_cb: New connection acceptance (required for listening sockets)

Structure Members:

- unsigned flags: Group flags (XLIO_GROUP_FLAG_$*$)
- xlio_socket_event_cb_t socket_event_cb: Socket event callback (required)
- xlio_socket_comp_cb_t socket_comp_cb: Zero-copy completion callback (optional)
- xlio_socket_rx_cb_t socket_rx_cb: Receive data callback (optional)
- xlio_socket_accept_cb_t socket_accept_cb: Accept callback for listening sockets (optional)

The documentation for this struct was generated from the following file:

- ultrapai.h

5.4 xlio_socket_attr Struct Reference

Socket creation attributes.

```
#include <ultrapai.h>
```

Public Attributes

- unsigned **flags**
- int **domain**
- xlio_poll_group_t **group**
- uintptr_t **userdata_sq**

5.4.1 Detailed Description

Socket creation attributes.

Structure containing parameters for socket creation with xlio_socket_create(). The socket is automatically associated with the specified polling group.

Domain Support:

- AF_INET: IPv4 support
- AF_INET6: IPv6 support

User Data:

- userdata_sq: Application-defined value for socket identification in callbacks
- Can be updated later with xlio_socket_update()

Structure Members:

- unsigned flags: Socket flags (reserved for future use)
- int domain: Address family (AF_INET or AF_INET6)
- xlio_poll_group_t group: Polling group to associate socket with
- uintptr_t userdata_sq: User data for socket identification in callbacks

The documentation for this struct was generated from the following file:

- ultrapai.h

5.5 xlio_socket_send_attr Struct Reference

Send operation attributes.

```
#include <ultrapai.h>
```

Public Attributes

- unsigned **flags**
- uint32_t **mkey**
- uintptr_t **userdata_op**

5.5.1 Detailed Description

Send operation attributes.

Structure containing parameters for send operations (xlio_socket_send/sendv). Controls zero-copy behavior, flushing, and completion tracking.

Zero-Copy Operation:

- mkey: Memory key for registered memory regions
- userdata_op: User data provided to completion callback
- For zero-copy, memory must be registered with ibv_pd from xlio_socket_get_pd()

Inline vs Zero-Copy:

- INLINE flag: Data copied to internal buffers, no completion callback
- Zero-copy: Data sent directly from user buffer, completion callback invoked

Structure Members:

- unsigned flags: Send flags (XLIO_SOCKET_SEND_FLAG_∗)
- uint32_t mkey: Memory key for zero-copy operation (ignored for inline)
- uintptr_t userdata_op: User data for completion callback (zero-copy only)

The documentation for this struct was generated from the following file:

- ultrapai.h

Chapter 6

File Documentation

6.1 ultrapai.h

```
00001
00086 /* Forward declaration. */
00087 struct ibv_pd;
00088
00110 int xlio_init_ex(const struct xlio_init_attr *attr);
00111
00128 int xlio_init(void);
00129
00137 int xlio_exit(void);
00138
// end of xlio_init group  00140
00184 int xlio_poll_group_create(const struct xlio_poll_group_attr *attr, xlio_poll_group_t *group_out);
00185
00195 int xlio_poll_group_destroy(xlio_poll_group_t group);
00196
00210 int xlio_poll_group_update(xlio_poll_group_t group, const struct xlio_poll_group_attr *attr);
00211
00224 void xlio_poll_group_poll(xlio_poll_group_t group);
00225
// end of xlio_poll_group group  00227
00259 int xlio_socket_create(const struct xlio_socket_attr *attr, xlio_socket_t *sock_out);
00260
00277 int xlio_socket_destroy(xlio_socket_t sock);
00278
00290 int xlio_socket_update(xlio_socket_t sock, unsigned flags, uintptr_t userdata_sq);
00291
00307 int xlio_socket_setsockopt(xlio_socket_t sock, int level, int optname, const void *optval,
00308                            socklen_t optlen);
00309
00322 int xlio_socket_getsockname(xlio_socket_t sock, struct sockaddr *addr, socklen_t *addrlen);
00323
00336 int xlio_socket_getpeername(xlio_socket_t sock, struct sockaddr *addr, socklen_t *addrlen);
00337
00354 int xlio_socket_bind(xlio_socket_t sock, const struct sockaddr *addr, socklen_t addrlen);
00355
00379 int xlio_socket_connect(xlio_socket_t sock, const struct sockaddr *to, socklen_t tolen);
00380
00401 int xlio_socket_listen(xlio_socket_t sock);
00402
00414 struct ibv_pd *xlio_socket_get_pd(xlio_socket_t sock);
00415
00434 int xlio_socket_detach_group(xlio_socket_t sock);
00435
00452 int xlio_socket_attach_group(xlio_socket_t sock, xlio_poll_group_t group);
00453
// end of xlio_socket group  00455
00500 int xlio_socket_send(xlio_socket_t sock, const void *data, size_t len,
00501                      const struct xlio_socket_send_attr *attr);
00502
00516 int xlio_socket_sendv(xlio_socket_t sock, const struct iovec *iov, unsigned iovcnt,
00517                       const struct xlio_socket_send_attr *attr);
00518
00531 void xlio_poll_group_flush(xlio_poll_group_t group);
00532
00547 void xlio_socket_flush(xlio_socket_t sock);
00548
// end of xlio_tx group  00550
00582 void xlio_socket_buf_free(xlio_socket_t sock, struct xlio_buf *buf);
```

```
00583
00595 void xlio_poll_group_buf_free(xlio_poll_group_t group, struct xlio_buf *buf);
00596
// end of xlio_rx group  00598
// end of xlio_ultra_api group  00600
00601
00602 XLIO Zerocopy API Data Structures
00614 typedef uintptr_t xlio_poll_group_t;
00615
00622 typedef uintptr_t xlio_socket_t;
00623
00648 struct xlio_buf {
00649     uint64_t userdata;
00650 };
00651
// end of xlio_rx group  00653
00684 typedef void (*xlio_memory_cb_t)(void *addr, size_t len, size_t hugepage_size);
00685
00687 enum {
00689     XLIO_SOCKET_EVENT_ESTABLISHED = 1,
00691     XLIO_SOCKET_EVENT_TERMINATED,
00693     XLIO_SOCKET_EVENT_CLOSED,
00695     XLIO_SOCKET_EVENT_ERROR,
00696 };
00697
00725 typedef void (*xlio_socket_event_cb_t)(xlio_socket_t sock, uintptr_t userdata_sq, int event,
00726                                        int value);
00727
00749 typedef void (*xlio_socket_comp_cb_t)(xlio_socket_t sock, uintptr_t userdata_sq,
00750                                       uintptr_t userdata_op);
00751
00771 typedef void (*xlio_socket_rx_cb_t)(xlio_socket_t sock, uintptr_t userdata_sq, void *data,
00772                                     size_t len, struct xlio_buf *buf);
00773
00791 typedef void (*xlio_socket_accept_cb_t)(xlio_socket_t sock, xlio_socket_t parent,
00792                                         uintptr_t parent_userdata_sq);
00793
// end of xlio_callbacks group  00795
00821 struct xlio_init_attr {
00822     unsigned flags;
00823     xlio_memory_cb_t memory_cb;
00824
00825     /* Optional external user allocator for XLIO buffers. */
00826     void *(*memory_alloc)(size_t);
00827     void (*memory_free)(void *);
00828 };
00829
// end of xlio_init group  00831
00838 #define XLIO_GROUP_FLAG_SAFE 0x1
00840 #define XLIO_GROUP_FLAG_DIRTY 0x2
00841
00864 struct xlio_poll_group_attr {
00865     unsigned flags;
00866
00867     xlio_socket_event_cb_t socket_event_cb;
00868     xlio_socket_comp_cb_t socket_comp_cb;
00869     xlio_socket_rx_cb_t socket_rx_cb;
00870     xlio_socket_accept_cb_t socket_accept_cb;
00871 };
00872
// end of xlio_poll_group group  00874
00900 struct xlio_socket_attr {
00901     unsigned flags;
00902     int domain; /* AF_INET or AF_INET6 */
00903     xlio_poll_group_t group;
00904     uintptr_t userdata_sq;
00905 };
00906
// end of xlio_socket group  00908
00915 #define XLIO_SOCKET_SEND_FLAG_FLUSH 0x1
00917 #define XLIO_SOCKET_SEND_FLAG_INLINE 0x2
00918
00939 struct xlio_socket_send_attr {
00940     unsigned flags;
00941     uint32_t mkey;
00942     uintptr_t userdata_op;
00943 };
00944
// end of xlio_tx group  00946
// end of xlio_ultra_api group
```

Index